Se publicó con Documentos de Google Denunciar abuso Más información

Taller 2 2026_01

Se actualiza automáticamente cada 5 minutos

UNIVERSIDAD DISTRITAL FRANCISCO JOSÉ DE CALDAS	**INGENIERÍA EN TELEMÁTICA CRIPTOLOGÍA**	V10. 2022

Taller #	2
Fecha	24/02/2026
Tema	Criptología de flujo
Objetivo	Reforzar conocimientos sobre los cifrados de flujo
Grupos	Sí máximo 3 personas
Fecha de entrega	

Parte 1

Los algoritmos Salsa20 y ChaCha20 han sido usado, incluso hasta el momento, en cifrado debido a su rapidez y baja carga computacional. En esta primera parte, observen las implementaciones para Salsa20 y Chacha20 que se encuentran a continuacón y que fueron obtenidas de [https://en.wikipedia.org/wiki/Salsa20]:

SALSA20:

```
#include <stdint.h>
#define ROTL(a,b) (((a) << (b)) | ((a) >> (32 -
(b))))
#define QR(a, b, c, d)(  \
        b ^= ROTL(a + d, 7), \
        c ^= ROTL(b + a, 9), \
        d ^= ROTL(c + b,13), \
        a ^= ROTL(d + c,18))
#define ROUNDS 20

void salsa20_block(uint32_t out[16], uint32_t const in[16])
{
        int i;
        uint32_t x[16];

        for (i = 0; i < 16; ++i)
                x[i] = in[i];
        // 10 loops × 2 rounds/loop = 20 rounds
        for (i = 0; i < ROUNDS; i += 2) {
                // Odd round
                QR(x[ 0], x[ 4], x[ 8], x[12]); //
column 1
                QR(x[ 5], x[ 9], x[13], x[ 1]); //
column 2
                QR(x[10], x[14], x[ 2], x[ 6]); //
column 3
                QR(x[15], x[ 3], x[ 7], x[11]); //
column 4
                // Even round
                QR(x[ 0], x[ 1], x[ 2], x[ 3]); //
row 1
                QR(x[ 5], x[ 6], x[ 7], x[ 4]); //
row 2
```

Se publicó con Documentos de Google Denunciar abuso Más información


```
}
```

CHACHA20

```
#define ROTL(a,b) (((a) << (b)) | ((a) >> (32 -
(b))))
#define QR(a, b, c, d) (             \
        a += b, d ^= a, d = ROTL(d, 16), \
        c += d, b ^= c, b = ROTL(b, 12), \
        a += b, d ^= a, d = ROTL(d,  8), \
        c += d, b ^= c, b = ROTL(b,  7))
#define ROUNDS 20

void chacha_block(uint32_t out[16], uint32_t const in[16])
{
        int i;
        uint32_t x[16];

        for (i = 0; i < 16; ++i)
                x[i] = in[i];
        // 10 loops × 2 rounds/loop = 20 rounds
        for (i = 0; i < ROUNDS; i += 2) {
                // Odd round
                QR(x[0], x[4], x[ 8], x[12]); //
column 1
                QR(x[1], x[5], x[ 9], x[13]); //
column 2
                QR(x[2], x[6], x[10], x[14]); //
column 3
                QR(x[3], x[7], x[11], x[15]); //
column 4
                // Even round
                QR(x[0], x[5], x[10], x[15]); //
diagonal 1 (main diagonal)
                QR(x[1], x[6], x[11], x[12]); //
diagonal 2
                QR(x[2], x[7], x[ 8], x[13]); //
diagonal 3
                QR(x[3], x[4], x[ 9], x[14]); //
diagonal 4
        }
        for (i = 0; i < 16; ++i)
                out[i] = x[i] + in[i];
}
```

Desarrollo:

Con base en las implementaciones anteriores, desarrollen implementaciones iguales para el cifrado y descifrado de mensajes usando ChaCha20 en el lenguaje de programación que escojan con las siguientes características:

- Para que todos los algoritmos sean iguales, cuando haya que usar caracteres o strings por favor codificar en utf8
- La salida de los programas, que será la que se ponga en los reportes, deberá mostrar cada una de las rondas usando el formato de matriz hexadecimal como se hace en la especificación de ChCha [RFC7536] en la sección 2.2 [https://datatracker.ietf.org/doc/html/rfc7539#section-2.2] (recomiendo observar éste documento completo si desean saber más sobre ChaCha y como se presentan las especificaciones de la RFC)
- Al final, de los programas se debe especificar la palabra cifrante, el texto cifrado y el estado final del “nonce”

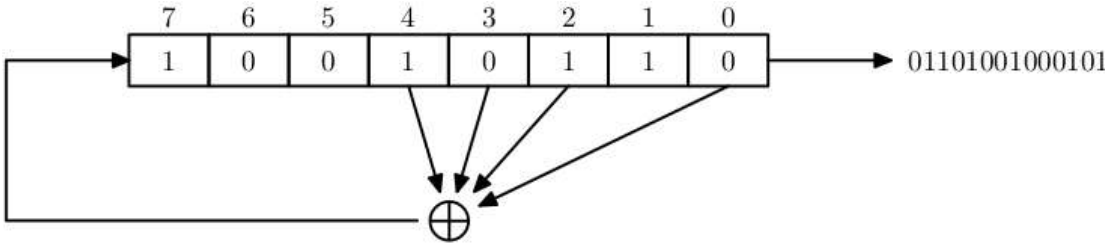
Se publicó con Documentos de Google Denunciar abuso Más información

2. Para observar el funcionamiento de la función QR, realicen el procedimiento de la sección 2.1.1 para el algoritmo de ChaCha
3. Codifique el mensaje (usando utf-8): "Este mensaje de prueba será cifrado con ChaCha20, un algoritmo de flujo rápido y seguro que usa una clave de 256 bits ahora." (Ojo: usar ceros para ajustar a bloques de 512 bits) usando como clave: “00:01:02:03:04:05:06:07:08:09:0a:0b:0c:0d:0e:0f:10:11:12:13:14:15:16:17:18:19:1a:1b:1c:1d:1e:1f” y como nonce 00:00:00:09:00:00:00:4a:00:00:00:00 y cuenta: 1. Verifiquen los resultados con la Sección 2.3.2 RFC 7539
4. Codificar el mismo mensaje, con la misma clave pero usando el nonce: “01:00:00:09:00:00:00:4a:00:00:00:00”
    a. ¿Desde qué ronda se observan cambios importantes en las matrices cambiando el nonce?
5. Repetir la prueba 1 pero cambiando la clave a: “01:01:02:03:04:05:06:07:08:09:0a:0b:0c:0d:0e:0f:10:11:12:13:14:15:16:17:18:19:1a:1b:1c:1d:1e:1f” (cambio de 1 bit en la clave)
    a. ¿Desde qué ronda se observan cambios importantes en las matrices cambiando un solo bit en la clave?
    b. ¿Los dos mensajes cifrados tienen algún parecido?
6. Repetir la prueba 1 pero cambiando el mensaje a: "Este mensaje de prueba será cifrado con ChaCha21, un algoritmo de flujo rápido y seguro que usa una clave de 256 bits ahora." (cambio de 1 bit en el mensaje)
    a. ¿Desde qué ronda se observan cambios importantes en las matrices cambiando un solo bit en el mensaje?
    b. ¿Los dos mensajes cifrados tienen algún parecido?

Parte 2 - LFSR

En los años 1980’s se diseñó un sistema de cifrado de flujo para tratar de evitar la piratería de las películas distribuídas en DVD el cual se llamó **Sistema de Desordenamiento de Contenidos** o **CSS** por sus siglas en inglés (*Content Scrambling System*). El sistema constaba de una clave de 40 bits (lo cual lo hace inseguro con los criterios actuales de mínimo 128 bits).

La base del cifrado está en registros de desplazamiento lineal retroalimentados (LFSR) los cuales se obtiene un bit de salida que es la combinación y corrimiento de n bits de entrada como se observa a continuación.



Como se puede observar el registro recibe una semilla y en cada ciclo obtiene un bit que depende de la semilla (clave) y la XOR de las posiciones “tap” en el ejemplo de la figura los *tap* son {0,2,3,4} (Pero pueden ser otros definidos por los usuarios). Una condición para que el LFSR funcione es que la semilla no sea solo 0’s.

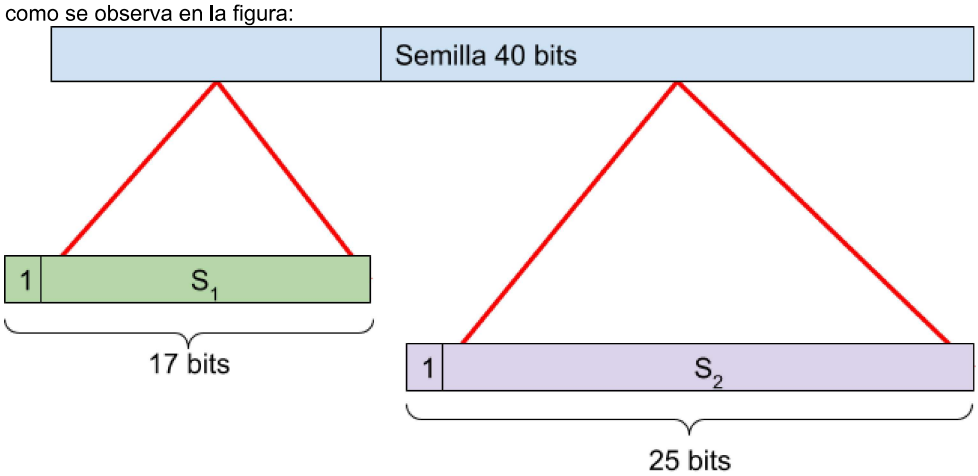
Cifrador CSS
El algoritmo se basa en dos filtros LFSR uno de 25 bits y el otro de 17 bits en la siguiente configuración:

Se publicó con Documentos de Google Denunciar abuso Más información

los registros LFSR tienen los tap en: S_1={14,0} y S_2={12,4,3,0}

De esta manera, el algoritmo hará lo siguiente:

1. Recibe la semilla (clave) de 40 bits $\{0,1\}^{40}$
2. Establece el bloque de salida de 8 bits
3. Envía los 40 bits a los registros fijando un 1 en el bit más significativo de cada registro, los 16 bits más significativos se enviarán al registro s_1 (17 bits) y los 24 bits menos significativos al registro s_2 (25 bits) como se observa en la figura:



4. Fija el bit de acarreo *c* en 0
5. Realiza un ciclo de 8 bits en ambos registros para obtener x_i y y_i y se pasan a entero mod 256
6. Se realiza la operación $z_i := (x_i + y_i + c) \bmod 256$
7. Si $x_i + y_i > 255 \rightarrow c := 1$ Si no c:= 0
8. Se repiten los pasos 5 a 7 por cada bloque de codificación necesario

De esta manera, se obtienen los bloques necesarios para realizar la XOR con el mensaje y así realizar la codificación,

Ejercicio para el informe:

1. Desarrollar un algoritmo CSS con los parámetros observados anteriormente y cifrar el siguiente mensaje de prueba (recordar usar utf-8)
   "El algoritmo CSS fue diseñado para proteger el contenido de los DVD mediante un cifrado de flujo basado en registros de desplazamiento. Aunque su seguridad fue superada, sigue siendo un hito clave en la historia de la gestión de los derechos digitales."
   y como clave: 0x1A:2B:3C:4D:5E (40 bits)

2. Realizar pruebas de avalancha de la clave y del mensaje, mostrando la cantidad de bits diferentes en cada caso.

Parte 3 - Algoritmo RC4

1. Usando los pseudoalgorítmos vistos en clase para el cifrado de flujo RC4 desarrolle un programa usando algún lenguaje de programación. Para esto, se tienen los siguientes parámetros:
   a. El diccionario el siguiente D = {ABCDEFGHIJKLMNÑOPQRSTUVWXYZ12345} (nótese que son 32 caracteres para 5 bits)

Se publicó con Documentos de Google Denunciar abuso Más información

Registro en informe: Adicionar al informe los codigos, el KeyStream en binario, la codificación en binario de 5 bits por caracter (la función xor del mensaje con el KeyStream) y el mensaje cifrado en el diccionario.

Pregunta para el informe: Al observar la clave de codificado en binario, ¿Cumple con los postulados de Golomb para ser pseudoaleatorio?

3. Codificar el mensaje usando una clave escogida por ustedes

   Registro en informe:

   a. Descifrar el mensaje usando la clave escogida y mostrar los resultados
   b. Descifrar el mensaje de este punto pero con la clave del punto 2 (CLAVE123) y mostrar los resultados

   Preguntas para el informe:

   a. ¿Fué posible distinguir de alguna manera el mensaje usando una clave incorrecta?
   b. ¿Sería posible hacer algún ataque estadístico usando una decodificación errónea?

Ataque de fuerza bruta

1. Usando el mismo algoritmo cambien el diccionario a D= {EAOLSNDRUITCPMYQ} (16 caracteres para 4 bits)
2. Codifiquen una palabra de 4 a 8 caracteres que tenga sentido en español, usando las letras del diccionario y con una clave de 4 caracteres (esta no necesita tener sentido en español), también del diccionario
3. Desarrollen la codificación del mensaje
4. Desarrollen un programa que pruebe todas las posibles combinaciones de claves de 4 caracteres y las presente en una tabla en donde aparezca cada posibilidad de clave y su mensaje decodificado y se pueda distinguir el mensaje correcto.

   Registro en informe:

   Registren el proceso de codificado, el código usado para el ataque de fuerza bruta y la lista de posibilidades obtenidas del ataque.

   Preguntas para el informe:

   a. Con ese diccionario y una clave de 4 caracteres ¿Cuántas posibilidades hay de claves? (respondan realizando el cálculo paso a paso)
   b. ¿Cuánto tiempo se demoró en obtener todas las posibilidades?
   c. ¿Dentro de las posibilidades aparecieron varias palabras con sentido?
   d. Como criptoanalistas ¿Qué criterio usarían para distinguir el mensaje correcto si salieran varios con sentido?
   e. ¿Qué estrategia usarían para reducir el tiempo de encontrar la clave correcta?

El formato del reporte es libre, solo debe contestar los puntos y las preguntas que se encuentran en este taller.

Informe:

Para el informe se debe adjuntar el archivo o enlace a la hoja de cálculo así como enlaces o los códigos usados acompañado de una descripción del proceso desarrollado.